P17 — Repo Restructuring & Onboarding Guide

> ***Date**: 2026-02-21 **Author**: kweng (via Claude Code) **Scope**: Repository structure alignment with `ai-project-base` template*

---

1. What Changed (Summary)

We restructured the repo to follow a clean **two-layer architecture**: a shared **base template** plus **project-specific code**. This brings us in line with how the `openclaw` fork is organized.

Key changes in this round:

- Merged latest `ai-project-base` (incremental scan, uncategorized replay, slug bug fix)
- Reverted locally-broken `.claude/commands/` files to match base
- Added `REPO_GUIDE.md` documenting file ownership
- Created spec P17 tracking this work

---

2. Repo Structure: Core (Public + RESTRICTED) + Base

The repo has **two layers**. Always edit in the layer that owns the file.

Layer Diagram

```
Enpack_CCC/
│
│  ═══════════════════════════════════════════════════
│  FROM BASE  (do NOT edit here — edit in ~/AI/base)
│  ═══════════════════════════════════════════════════
├── PROJECT_GUIDELINES.md          # Workflow conventions
├── WORK_LOG.md                    # Session work log
├── specs/                         # Base spec infrastructure
│   ├── 00_template/               #   Spec templates (requirements, design, tasks,
status)
│   └── README.md                  #   Spec index
├── src/session_history/           # Session history tool (Python)
├── .claude/commands/log.md        # /log command
├── .claude/commands/history.md    # /history command
├── .claude/skills/log.md          # /log skill
├── .claude/skills/history.md      # /history skill
├── .claude/skills/create-task.md  # /create-task skill
│
│  ═══════════════════════════════════════════════════
│  PROJECT-SPECIFIC  (edit here, push to origin)
│  ═══════════════════════════════════════════════════
├── REPO_GUIDE.md                  # File ownership documentation
├── README.md                      # Project README
├── .session-history.json          # Entity config for session_history
├── .claude/settings.json          # Hooks (extends base with project hooks)
├── .claude/commands/process.md    # /process command (doc processing)
├── .gitignore                     # Extends base with project patterns
```

```
│
│  ── Public Specs (P## prefix) ──
├──    /
│   ├── P01_          /
│   ├── P02_                    /
│   ├── ...
│   ├── P13_               /
│   ├── P16_             /
│   └── P17_          Base/
│
│  ── Restricted Specs (R## prefix) ──
├── RESTRICTED/
│   └──    /
│       └── R14_               /
│
│  ── AI Subsidiary Specs (A## prefix) ──
├── AI     /                         # Private repo (gitignored)
│   └──    /
│       └── A15_       AI                  /
│
│  ── Source Code ──
├──      /
│   ├── chunked_processor/         # P13: Large doc chunking
│   ├── doc_indexer/               # Document indexing
│   ├── experiment_analyzer/       # Experiment analysis
│   └── session-persistence/       # P03: Session capture (JS, legacy)
│
│  ── Knowledge & Research ──
├──      /                         # Knowledge base
├──    /                           # Research notes
├──    /                           # Documentation
│
│  ── Session History (auto-generated) ──
└──       /                       # Centralized history storage
    ├── all-sessions.json          # Master session index
    ├── categorization-report.md   # Classification stats
    ├── .scan-state.json           # Incremental scan state
    ├── uncategorized/             # Sessions that didn't match any entity
    │   └── sessions.json
    └── entities/                  # Centralized entity histories
        ├── source/<name>/         #   e.g. source/chunked_processor/
        ├── research/<name>/       #   e.g. research/        /
        └── knowledge/<name>/      #   e.g. knowledge/01_       /
```

The Key Rule

> ***Base-owned files → edit in `~/AI/base`, then pull via `git fetch base && git merge base/main`. Everything else → edit directly in this repo.***

How Specs Are Organized

Prefix	Location	Visibility	Examples

P##	/P##_name/	Public (in main repo)	P01–P13, P16, P17
R##	RESTRICTED/ /R##_name/	Restricted (sensitive)	R14
A##	AI / /A##_name/	AI subsidiary (private)	A15

Each spec directory has:

```
   /P13_               /
├── requirements.md    # What needs to be done
├── design.md          # How it will be done
├── tasks.md           # Breakdown of work items
├── status.md          # Implementation progress
└── history/           # ← Session history lives here (inline)
    ├── sessions-index.json
    ├── replay-index.md
    └── replay/
        ├── kweng_2026-02-21_01-37.md
        └── kweng_2026-02-19_20-53.md
```

How History Is Organized

The session history tool uses **two storage strategies**:

Entity Type	Storage	Location
Spec (P##, R##, A##)	Inline	/P##_name/history/, RESTRICTED/ /R##_name/history/, or AI / /A##_name/history/
Task	Inline	tasks/##_name/history/
Source, Research, Knowledge, Tool	Centralized	/entities/<type>/<name>/
Uncategorized	Centralized	/uncategorized/

"Inline" means the history lives inside the entity's own directory — so a spec's session history is right next to its design docs. "Centralized" means it goes to the / directory tree.

Note on specs/ vs /

You'll notice two spec-like directories at the root:

Directory	Owned by	Contains
specs/	**Base**	Only templates (00_template/) and README.md
/	**Project**	All actual project specs (P01–P17)

The specs/README.md from base says "Create directory: ##_descriptive-name/" as if you'd put specs inside specs/, but in this project we put them in / instead. This is fine — the README is a generic template, and each project adapts the convention to its own directory.

Bottom line: no action needed. The current split (`specs/` = base template, `/` = project specs) is the intended design and matches the openclaw pattern (`specs/` = base template, `spec/` = project specs). The session history tool auto-discovers specs from all three possible directories (`/` , `spec/` , `specs/`), so classification works regardless of which naming convention a project uses.

---

3. Sample History Session File

File naming convention

```
{username}_{date}_{time}.md
```

Example: `kweng_2026-02-21_01-37.md`

Sample replay file content

```
# Spec 17:           Base - Session Replay

## Session: 2026-02-21T01:37 ~ 01:46
> Person: kweng | Messages: 176 | Turns: 1

---

### 01:37 - 1. look at ~/AI/openclaw repo structure, it has a base...

**Prompt:**
> 1. look at ~/AI/openclaw repo structure, it has a base project,
> 2. develop a new spec to ...

<details>
<summary>Full prompt (700 chars)</summary>
[full text of the user's prompt]
</details>

**Result:**
I'll start by exploring both repo structures and the current codebase...

**Tools used:**
- Task (×3)
- Bash (×8)
- Read (×4)
- Write (×5)
- Edit (×2)

---

### 01:42 - Can you also check the .gitignore...

**Prompt:**
> Can you also check the .gitignore for overlap?
```

```
**Result:**
[assistant's response summary]

**Tools used:**
- Bash (×2)
- Grep (×1)
```

Supporting index files

`replay-index.md` — links to all replay files for an entity:

```
# Spec 17:           Base - Replay Index

> Generated: 2026-02-21 09:46
> Sessions: 1

| Date | Person | File |
|------|--------|------|
| 2026-02-21_01-37 | kweng | [kweng_2026-02-21_01-37.md](replay/kweng_2026-02-21_01-37.md) |
```

`sessions-index.json` — machine-readable index with confidence scores, timestamps, message counts.

---

4. How to Get Latest Remote Changes

Step 1: Fetch and check what's new

```
cd ~/AI/Enpack_CCC

# See what's new on origin
git fetch origin
git log --oneline HEAD..origin/main

# See what's new on base
git fetch base
git log --oneline HEAD..base/main
```

Step 2: Stash or commit your local work

```
# Option A: Commit your work first (preferred)
git add <your-files>
git commit -m "WIP: your work description"

# Option B: Stash if not ready to commit
git stash
```

Step 3: Merge remote changes

```
# Merge origin (teammate's changes)
git merge origin/main

# If there are conflicts:
#   1. Open conflicted files, look for <<<<<<< / ======= / >>>>>>> markers
#   2. Resolve by keeping the correct version
#   3. git add <resolved-files>
#   4. git commit (to complete the merge)
```

Step 4: Merge latest base (if needed)

```
git fetch base
git merge base/main
# This brings in base template improvements (session_history, commands, etc.)
```

Step 5: Push safely

```
# Always pull before pushing to avoid force-push situations
git pull origin main --no-rebase
git push origin main
```

If you get "rejected (non-fast-forward)"

This means someone pushed while you were working. Do NOT force push. Instead:

```
git pull origin main --no-rebase   # merge their changes into yours
# resolve any conflicts if needed
git push origin main               # now push
```

---

5. Using `/history` — Scan, Replay & Review

All session history operations are done through the `/history` command in Claude Code. No need to run `python3` manually — `/history` handles everything.

Available Commands

Command	What it does
`/history scan`	Incremental scan — classify new/modified sessions
`/history scan` (say "full scan")	Full re-scan — reprocess all sessions
`/history list`	List all sessions with their classifications
`/history replay P17`	Generate readable Markdown replay for an entity
`/history search REPO_GUIDE`	Search across all session content
`/history stats`	Show classification statistics overview

`/history` (no args)	Defaults to `list`

Typical Workflow

1. **`/history scan`** — Classify sessions (run this first, or after any repo changes)
2. **`/history stats`** — Check the classification rate (should be high)
3. **`/history replay P17`** — Generate replay for a spec you care about
4. **Open `replay-index.md`** in the entity's `history/` dir — it links to all replay files
5. **Read the `.md` replay files** — each is a readable transcript organized by turn (user prompt → assistant response → tools used)

Sample Output

`/history stats`:

```
      : 3
    :   [████████████████████] 100.0%
    : 3 |      : 0
```

`/history list`:

```
  272927c6... | 2026-02-21 |  72 msgs
    → Spec 15:        AI                   (0.30)

  4ff8dafa... | 2026-02-21 | 166 msgs
    → Spec 17:           Base (0.18)
```

Verifying History After Repo Changes

After any restructuring or base merge, verify with this sequence:

1. `/history scan` — say "full scan" to reprocess everything
2. `/history stats` — classification rate should be high
3. `/history replay P17` — check that replay files are generated
4. Open `    /P17_          Base/history/replay-index.md` — spot-check content

How `/history` Works Under the Hood

```
Claude Code session
  → saved as JSONL at ~/.claude/projects/-Users-kweng-AI-Enpack-CCC/*.jsonl
  → /history scan reads these JSONL files
  → classifier matches sessions to entities (specs, source, research, etc.)
    using 3 signals: file paths (30%), text patterns (40%), keywords (30%)
  → sessions-index.json written to each matched entity's history/ dir
  → /history replay reads the index + JSONL → generates readable .md files
```

The entity auto-discovery reads your directory structure:

- `    /P##_name/` → detected as Spec entities
- `RESTRICTED/    /R##_name/` → detected as Spec entities (restricted)
- `AI     /    /A##_name/` → detected as Spec entities (AI subsidiary)
- `      /<name>/` → detected as Source entities
- `    /<name>/` → detected as Research entities

- `/<name>/` → detected as Knowledge entities

Configuration is in `.session-history.json` at the project root. The underlying tool lives at `src/session_history/` (from base).

What `/history` Does NOT Do

- It does NOT capture sessions in real-time (Claude Code does that automatically as JSONL)
- It does NOT modify session data — it only reads and classifies
- It does NOT delete anything — re-running scan just updates the indices

---

6. Quick Checklist for New Team Members

- [ ] Clone the repo and `cd Enpack_CCC`
- [ ] Add base remote: `git remote add base https://github.com/kevinw99/ai-project-base.git`
- [ ] Read `REPO_GUIDE.md` for file ownership rules
- [ ] Read `PROJECT_GUIDELINES.md` for workflow conventions
- [ ] In Claude Code, run `/history scan` to build initial session indices
- [ ] Run `/history stats` to see current classification state
- [ ] Before editing any file, check: is it base-owned? If yes, edit in `~/AI/base` instead

---

Questions? Check `REPO_GUIDE.md` or ask in the team chat.